Name: _____

Student Id: _____

1	
2	
3	
4	
5	
6	
7	
Total	

CS590 Final exam

Question 1 [10 points] {5 minutes}

Give 6 different and valid issues/challenges we face when we implement a microservice architecture.

Answer:

1. Complex Communication
2. Data and Transaction Consistency
3. Resilience and Network Reliability
4. Security Complexity
5. Monitoring and Observability
6. Syncing Interfaces and Configuration

If your answer does not fit, continue at the back of this page

Question 2 [15 points] {15 minutes}

Suppose you need to decide between a component based monolith (also called a modular monolith) or a microservice architecture for a certain application. For each of the requirements given below indicate which architectural style would you use to fulfill this requirement.

Your answer can be one of the following 3 possibilities:

1. Component based monolith fits better

2. Microservice architecture fits better

3. Both architectures can be used to fulfill this requirement.

a. It should be easy to add new functionality. A: **Both architectures can be used to fulfill this requirement**

b. The architecture should be simple. A: **Component based monolith fits better**

c. It should be easy to try new technology. A: **Microservice architecture fits better**

d. The performance should be very fast. A: **Component based monolith fits better**

e. It should be easy to reuse existing functionality. A: **Both architectures can be used to fulfill this requirement**

f. Deployment should be very fast. A: **Microservice architecture fits better**

g. It should be easy to scale different functionalities differently. A: **Microservice architecture fits better**

h. It should be easy to use different types of databases. A: **Both architectures can be used to fulfill this requirement.**

i. The scope of the functionality is large. A: **Microservice architecture fits better**

j. We need multiple agile/scrum teams to implement and maintain the functionality. A: **Both architectures can be used to fulfill this requirement.**

If your answer does not fit, continue at the back of this page

Question 3 [15 points] {15 minutes}

Circle all statements that are correct:

1. **A custom router in Spring Integration can return a list of Strings**
2. A domain service object is a Spring bean annotated with @Service
3. **A repository in a Spring Boot application is implemented as an interface.**
4. In MongoDB you do not want data redundancy because this makes database actions slow.
5. **In MongoDB you do not want joins between collections because they are very slow.**
6. **In a Service Oriented Architecture the registry functionality is implemented in the ESB**
7. In a modular monolith the components communicate with each other using REST or messaging
8. **A value object in domain driven design is immutable**
9. DTO classes reside in the domain layer
10. **Partitioning and replication can be used at the same time**
11. In a relational database you first find the queries you need for your application and then design the tables based on these queries.
12. In a MongoDB database you first design the database collections and then write the queries in the application to get the required data.
13. **In a Cassandra database you first find the queries you need for your application and then design the tables based on these queries.**
14. The registry in a microservice architecture contains a client side load balancer.
15. **The API gateway in a microservice architecture contains a client side load balancer.**

If your answer does not fit, continue at the back of this page

Question 4 [10 points] {15 minutes}

Suppose you need to design an architecture for a webshop with the following architectural requirements:

- You need 1 agile devops team to implement the customers functionality
- You need 1 agile devops team to implement the orders functionality
- You need 1 agile devops team to implement the products functionality
- You need 1 agile devops team to implement the shoppingcart functionality
- You need 1 agile devops team to implement the payments functionality
- You need 1 agile devops team to implement the reviews functionality
- You need 1 agile devops team to implement the shipping functionality
- You need to be able to scale customers functionality differently than other functionality
- You need to be able to scale products functionality (and everything that is related to products) differently than other functionality
- You need strict consistency between the order data, payment data and shipping data

Explain clearly how you would design your architecture. Explain ALL architectural decisions that you need to make to fulfill the given requirements. For every architectural decision you make indicate how this decision solves one or more of the given requirements.

Answer:

- A microservice architecture with independent services for Customers, Products, Reviews, and Shoppingcart.
- Modular monolithic with Orders, Payments, and Shipping using a shared relational database.

If your answer does not fit, continue at the back of this page

Question 5 [25 points] 50 minutes}

Suppose you need to design the class diagram of an airline application with the following requirements

Customers can browse flights. A flight has an origin and a destination. The origin and destination is recorded in the system with the attributes airport abbreviation, airport name and airport country. For a flight we also keep track of the flight number and the total duration of the flight.

For a flight we also need to know the plane name, number of seats on the plane and the maximum distance this plane can fly.

Flights are offered one or more times a day. For every flight offer we need to know the date and time of the flight and the gate number and status of the gate.

Customers can buy tickets, and for a ticket they can choose the flight, the seat and also their meal preference. For every ticket we also need to keep track of the type of ticket which can be economy class, first class or business class.

For a flight we also need to keep track of the seat availability of the flight.

Customers can create an account so they can login with their username and password.

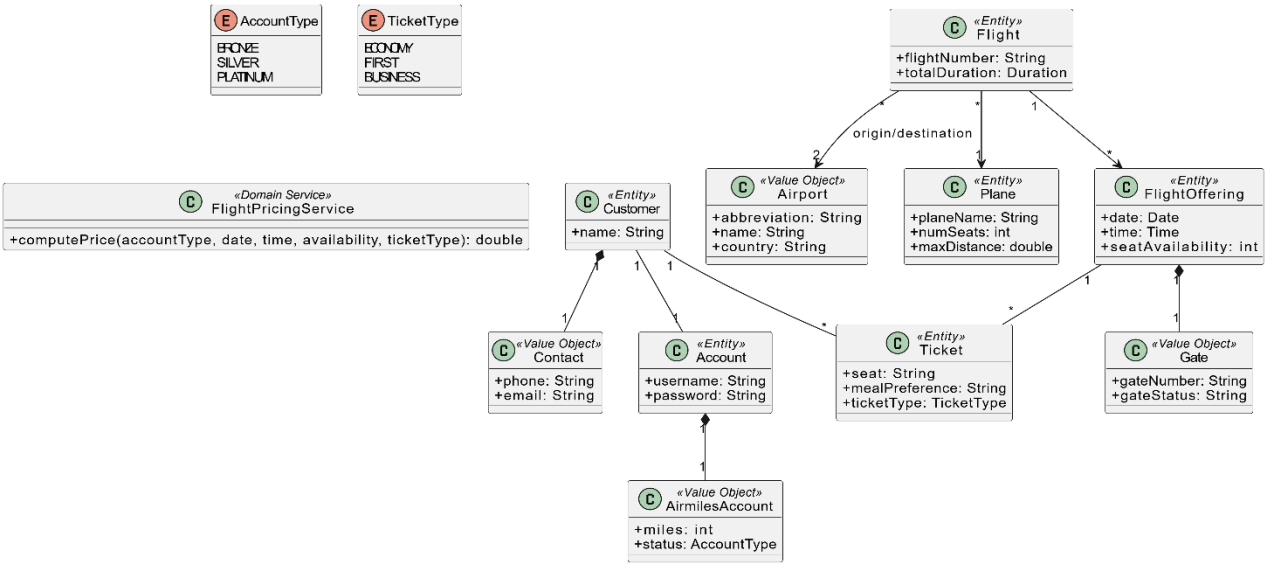
The account also keeps track of the airmiles this customer has earned. If the customer creates a new account, then this account is by default a bronze account. When the customer has enough airmiles, it is upgraded top a silver account. There is also a platinum account. For every customer we need to keep track of the name, phone number and email address.

When a customer searches for flights, the system dynamically computes the price for this flight. The prize for a flight depends on: Customer account type, date of the flight, time of the flight and seat availability of the flight and ticket type.

a. [10 points] Suppose we implement this application as one large monolith (without components). Draw the class diagram of the domain classes of this application. For every domain class, indicate its type (entity, value object, etc). Important: only draw the domain classes. Make sure you show the classes, relationships, attributes and methods (if applicable).

Draw your answer on the next page

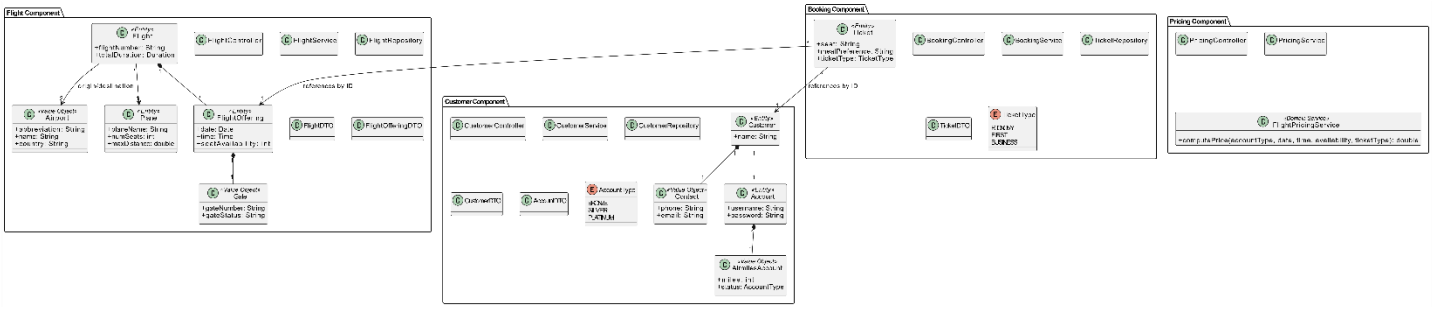
If your answer does not fit, continue at the back of this page



If your answer does not fit, continue at the back of this page

b. [25 points] Suppose we implement this application as a monolith with components, also called a modular monolith. Draw the class diagrams of all components. Important: draw ALL classes for every individual component. For every domain class, indicate its type (entity, value object, etc). For every domain class show the classes, relationships, attributes and methods (if applicable). For all NON domain classes only show the classes (not the relationships , methods and attributes)

Answer



If your answer does not fit, continue at the back of this page

Question 6 [10 points] {10 minutes}

Suppose we have the following relational database tables:

courseNumber	courseName
CS590	Software Architecture
CS525	Advanced Software Development

studentid	firstName	lastName
5566	Frank	Brown
5532	John	Doe

studentid	courseNumber	grade
5566	CS590	A+
5566	CS525	B
5532	CS590	A-

Because the data we have does not fit on 1 node anymore we decided to replace our relational database with a Cassandra database. It is an important requirement that the application that uses this database should give the same fast performance that we had when we used the relational database.

The application has 2 main pages. The first page allows us to see all grades, including the course information, for a certain student. The second page allows us to see all grades, including the student information, from a certain course. Show the structure of the table (or tables) that we need in Cassandra including the data in the table(s). For every table, specify what column is the partition key.

Answer:

If your answer does not fit, continue at the back of this page

Grades-per-student table

studentId in partition key

studentid	firstname	lastname	coursenumber	coursename	grade
5566	Frank	Brown	CS590	Software architecture	A+
5566	Frank	Brown	CS525	Advanced Software Development	B
5532	John	Doe	CS590	Software architecture	A-

Grades-per-course table

courseNumber is partition key

coursenumber	coursename	grade	studentid	firstname	lastname
CS590	Software architecture	A+	5566	Frank	Brown
CS590	Software architecture	B	5532	John	Doe
CS525	Advanced Software Development	B	5566	Frank	Brown

Question 7 [5 points] {10 minutes}

Describe how an Enterprise Service Bus (ESB) relates to one or more of the SCI principles you know. Your answer should be about 2 paragraphs. The number of points you get for this question depends on how well you explain the relationship between a ESB and the principles of SCI.

Answer:

In a Service Oriented Architecture (SOA), the ESB serves as a central orchestrator where business processes run to coordinate all activities across the system. This role relates to the SCI principle stating that "the more you are in tune with the laws of nature, the more support of nature you are able to enjoy".

Additionally, the ESB enables loose coupling through messaging integration patterns, allowing separate applications to communicate without being tightly bound to one another. This reflects the SCI concept of the "Unified Field," where "the whole relative creation is an expression of the same one unified field".

If your answer does not fit, continue at the back of this page